

向量
位图：数据结构

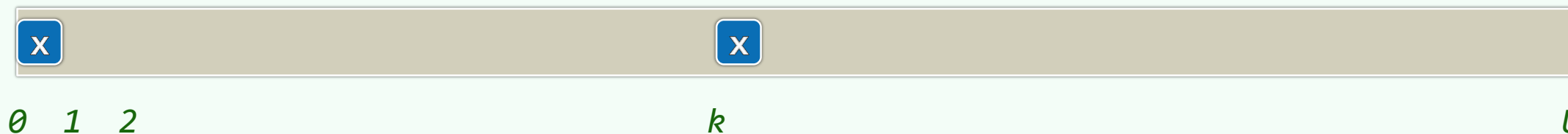
有限整数集

$\forall 0 \leq k < U :$

$k \in S ?$ `bool test( int k );`

$S \cup \{k\}$ `void set( int k );`

$S \setminus \{k\}$ `void clear( int k );`



结构

❖ class Bitmap {

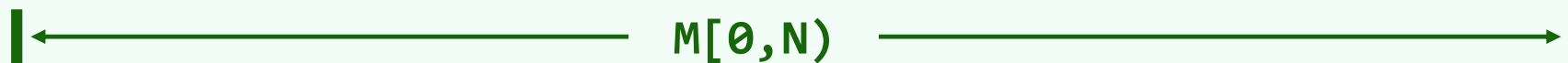
private:



int N;



char * M;



public:

Bitmap(int n = 8) { M = new char[N = (n+7)/8]; **memset**(M, 0, N); }

~Bitmap() { delete [] M; M = NULL; }

void **set**(int k); void **clear**(int k); bool **test**(int k);

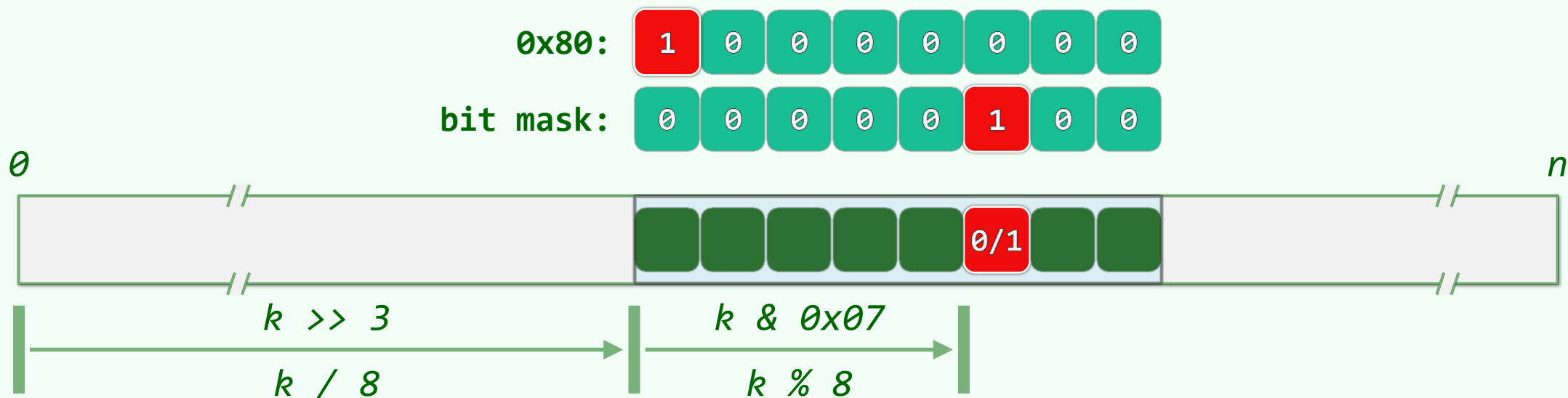
};

实现

❖ `bool test( int k ) { return M[  $k \gg 3$  ] & (  $0x80 \gg (k \ \& \ 0x07)$  ); }`

❖ `void set( int k ) { expand( k ); M[  $k \gg 3$  ] |= (  $0x80 \gg (k \ \& \ 0x07)$  ); }`

❖ `void clear( int k ) { expand( k ); M[  $k \gg 3$  ] &= ~(  $0x80 \gg (k \ \& \ 0x07)$  ); }`



向量
位图：典型应用

小集合 + 大数据：问题

❖ 老问题：int A[n]的元素均取自[0, m)

如何剔除其中的重复者？

❖ 仿照Vector::deduplicate()改进版

先排序，再扫描

—— $O(n \log n + n)$ —— 毫无压力

❖ 新特点：数据量虽大，但重复度极高

- 想想我们电脑里的mp3、mp4
- 还有，朋友圈 ...

❖ 比如：

$$2^{24} = m \ll n = 10^{10}$$

亦即，10,000,000,000个24位无符号整数

❖ 如果采用内部排序算法

至少需要 $4 * n = 40GB$ 内存

——否则，频繁的I/O将导致整体效率的低下

❖ 那么

$m \ll n$ 的条件，又应如何加以利用？

小集合 + 大数据：算法

❖ Bitmap B(m); //O(m)

```
for (int i = 0; i < n; i++)
```

```
    B.set( A[i] ); //O(n)
```

```
for (int k = 0; k < m; k++)
```

```
    if ( B.test( k ) )
```

```
        /* ... */; //O(m)
```

❖ 拓展：搜索引擎的使用规律亦是如此

词表**规模**不大，但**重复度**极高

——如何从中剔除重复的索引词？

❖ 总体运行时间 = $O(n+m) = O(n)$

❖ 空间 = $O(m)$

- 就上例而言，降至：

$$m/8 = 2^{21} = 2\text{MB} \ll 40\text{GB}$$

- 即便 $m = 2^{32}$ ，也不过：

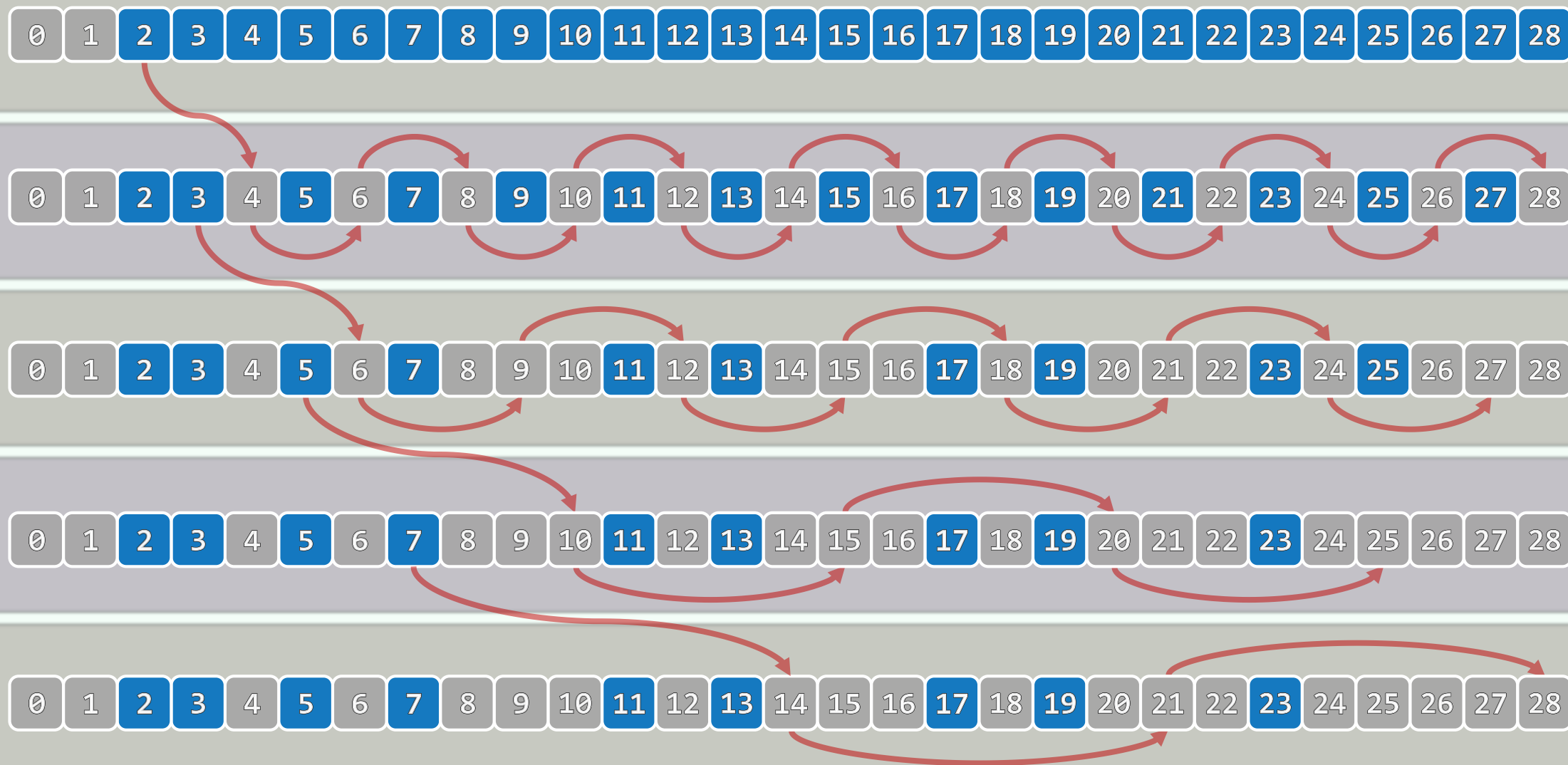
$$2^{29} = 0.5\text{GB}$$

❖ 关键在于

如何将查询词表转换为某一集合

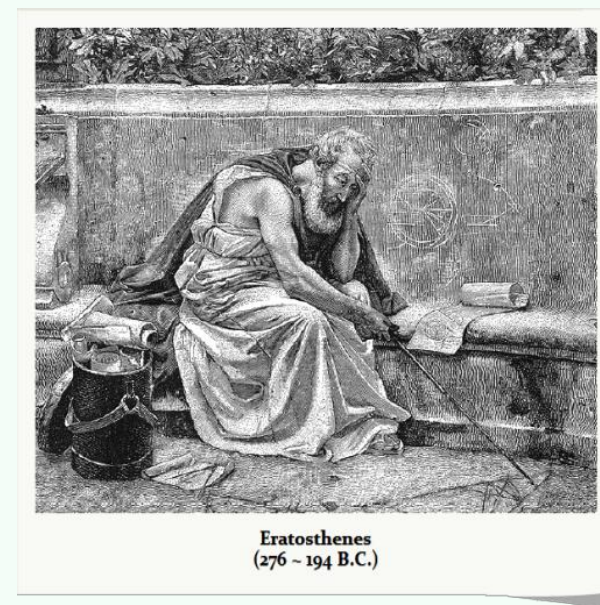
——留作习题

筛法：思路

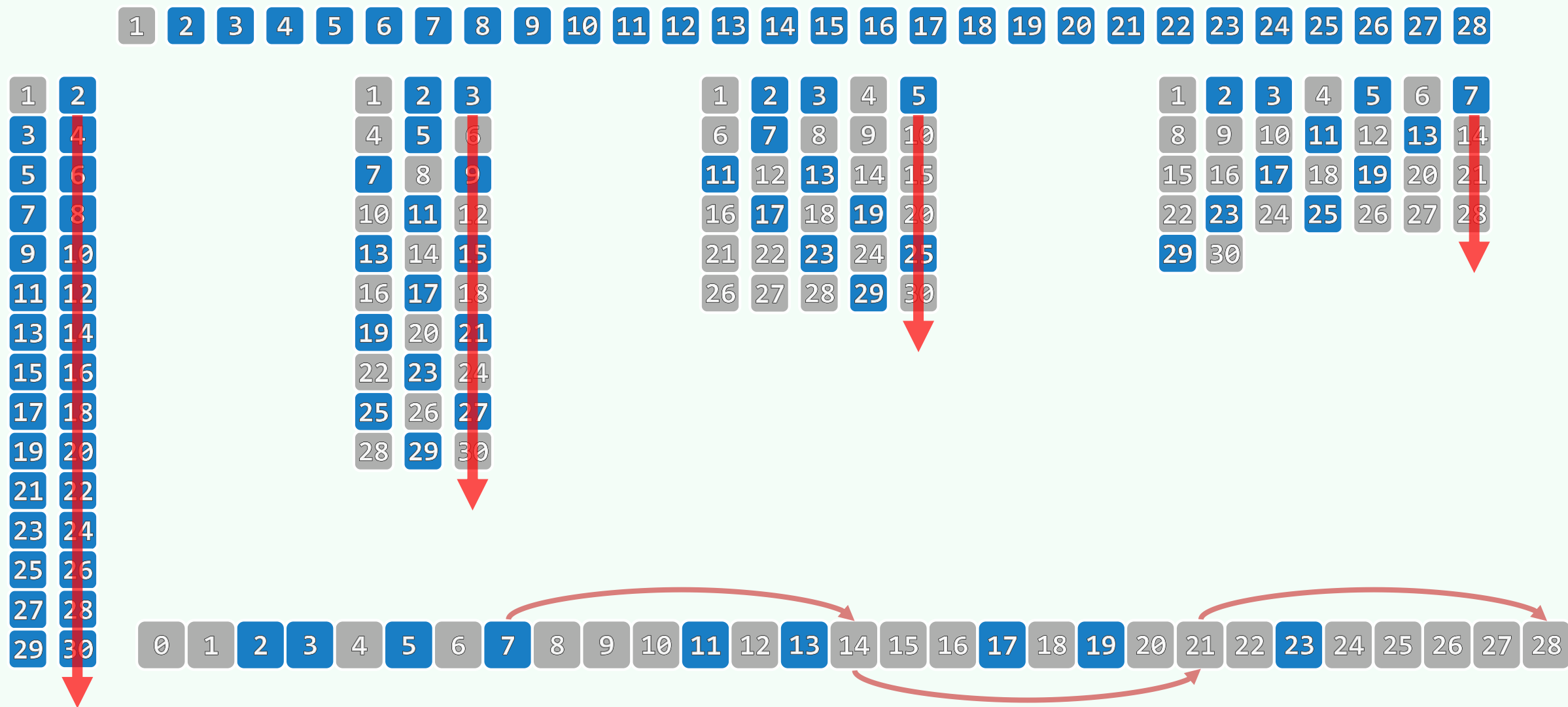


筛法：实现

```
❖ void Eratosthenes( int n, char * file ) {  
    Bitmap B( n );  
    B.set( 0 ); B.set( 1 );  
    for ( int i = 2; i < n; i++ )  
        if ( ! B.test( i ) )  
            for ( int j = 2*i; j < n; j += i )  
                B.set( j );  
    B.dump( file );  
}
```



筛法：过程 + 效果



向量
位图：快速初始化

$$O(n) \sim O(1)$$

❖ Bitmap的构造函数中，通过`memset(M, 0, N)`统一清零

这一步只需 $O(1)$ 时间？不，实际上仍等效于诸位清零， $O(N) = O(n)!$

❖ 尽管这并不会影响上例的渐近复杂度，但并非所有问题都是如此

❖ 有时，对于大规模的散列表（第09章），初始化的效率直接影响到实际性能

例如：第13章中`bc[]`表的构造算法，需要 $O(|\Sigma| + m) = O(s + m)$ 时间

若能省去`bc[]`表各项的初始化，则可严格地保证是 $O(m)$

❖ 有时，甚至会影响到算法的**整体**渐近复杂度

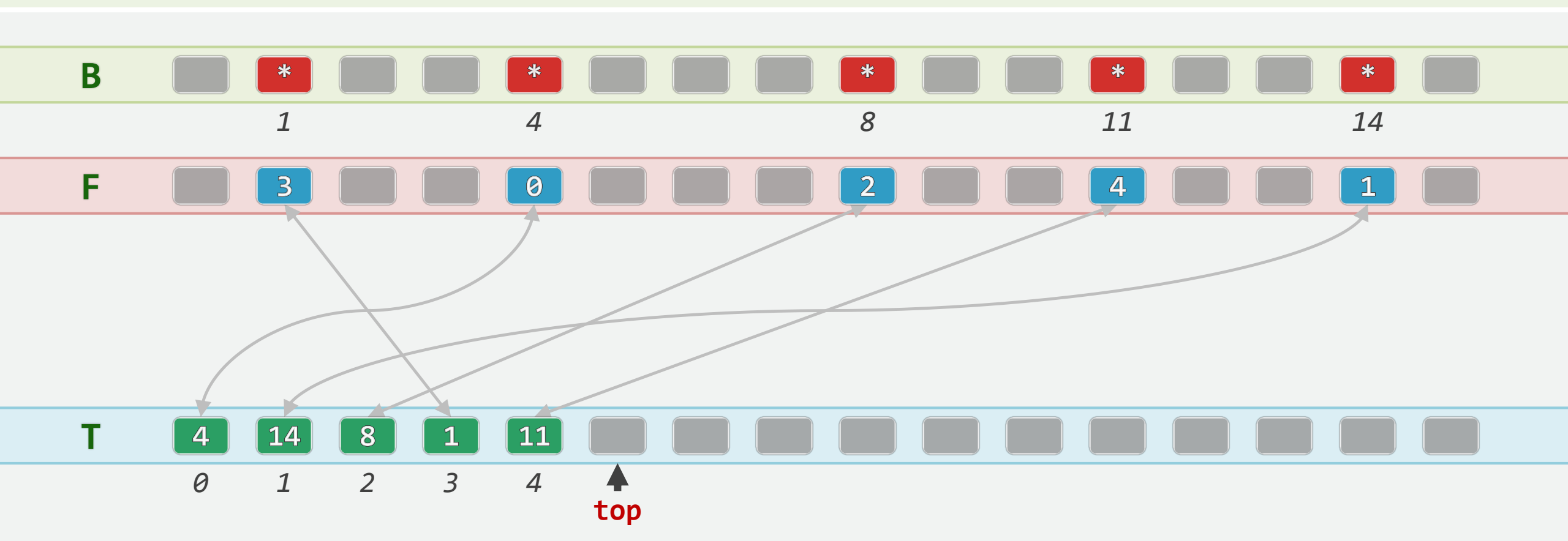
例如，为从 $n=10^8$ 个32位整数中找出重复者，可仿造**剔除**算法...

//但这里**无需回收**

因此，若能省去Bitmap的初始化，则只需 $O(n)$ 时间

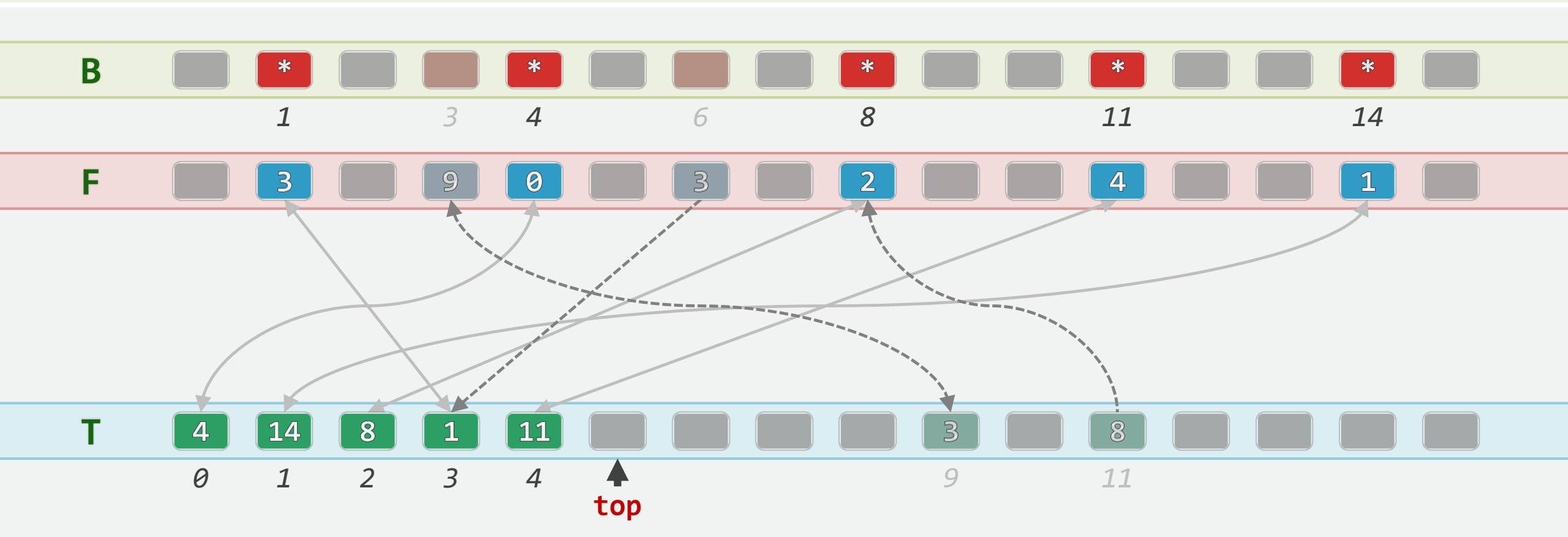
J. Hopcroft, 1974: **B[]**拆分成一对等长向量: Rank **F[m]**, **T[m]**, **top** = 0;

❖ 构成校验环: $T[F[k]] == k$ & $F[T[i]] == i$



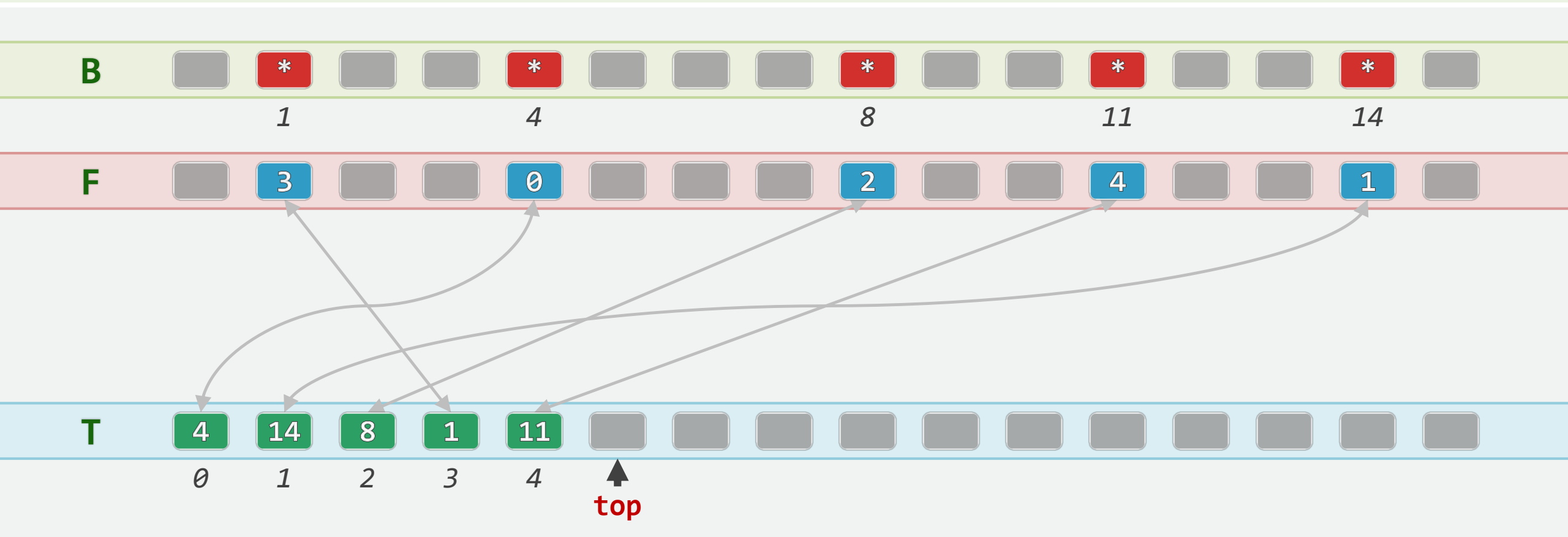
测试: $\text{test}(1,4,8,11,14) = \text{true}$ & $\text{test}(3,6) = \text{false}$

```
bool Bitmap::test( Rank k ) { return (0 <= F[k]) && (F[k] < top) && (k == T[F[k]]); }
```



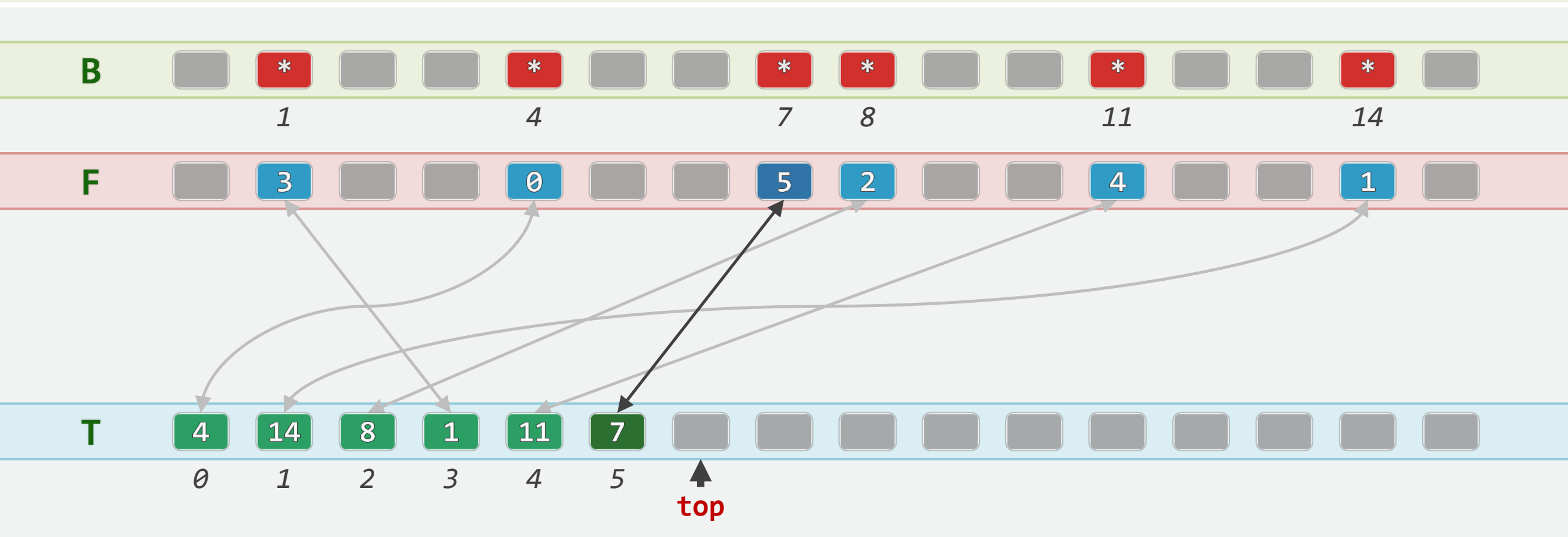
$O(1)$ 复位

```
void Bitmap::reset() { top = 0; }
```



$O(1)$ 插入: set(7)

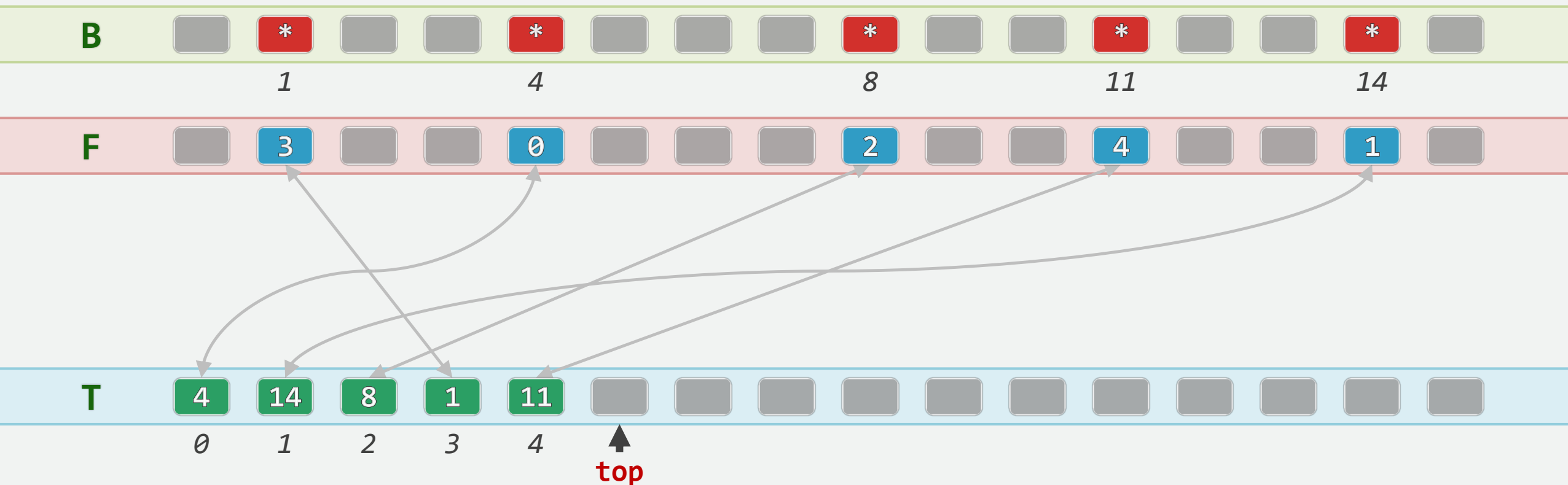
```
void Bitmap::set( Rank k ) { if ( !test( k ) ) { T[top] = k; F[k] = top++; } }
```



$O(1)$ 删除: remove(7)

```
void Bitmap::clear( Rank k )
```

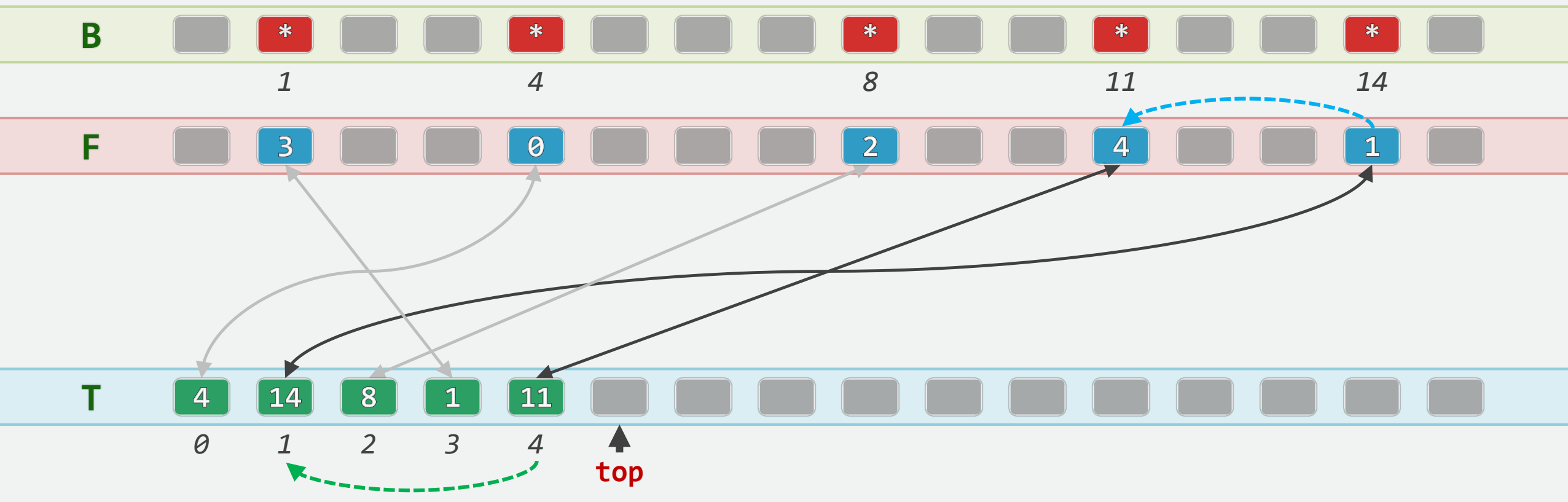
```
{ if ( test( k ) && ( --top ) ) { F[T[top]] = F[k]; T[F[k]] = T[top]; } }
```



$O(1)$ 删除: remove(14) ...

```
void Bitmap::clear( Rank k )
```

```
{ if ( test( k ) && ( --top ) ) { F[T[top]] = F[k]; T[F[k]] = T[top]; } }
```



$O(1)$ 删除: ... remove(14)

```
void Bitmap::clear( Rank k )
```

```
{ if ( test( k ) && ( --top ) ) { F[T[top]] = F[k]; T[F[k]] = T[top]; } }
```

